FlowPilot AI — PRD

Oct 19, 2026 · @Bnaybir

Overview

FlowPilot AI is an email agent for property managers: it reads tenant emails, looks up the tenant, unit and lease, drafts a reply or a repair ticket, and waits for a human to approve before anything is sent.

- **Problem:** a property manager with hundreds of units gets dozens of tenant emails a day (repairs, payments, lease questions). Reading each one, finding the lease and replying takes hours.
- **Target user:** one property manager at one company (single user for the MVP).
- **Why this project:** a portfolio piece that matches the Agents, Integrations and Product work at agent-heavy SaaS teams such as Hausverwalter.ai: a production-style agent, background queues, long-running workflows and a database designed from scratch, on the same stack (TypeScript, Convex, Next.js, shadcn/ui).
- **Success:** on the live demo, a test email arrives, the agent handles it in under a minute, and a human approves the reply from the dashboard.

Scope

The MVP is email only: five features, everything else is out.

Feature	What it does
Email inbox agent	Reads each new tenant email, classifies it, gathers context, drafts a reply or ticket
Human approval	Every outbound email and every new ticket waits for Approve, Edit or Reject
Bulk import	Imports thousands of buildings, units and tenants from CSV in the background
Knowledge base	Lease PDFs are extracted, chunked and searchable by the agent

Feature	What it does
Dashboard	Inbox, live agent run timeline, approvals queue, properties, import jobs

Out of scope: WhatsApp, phone and voice, video calls, team chat, multiple companies, payments, mobile app, customers building their own agents.

Tech stack

Everything is TypeScript and every piece has a free tier, so the MVP costs nothing to build or host.

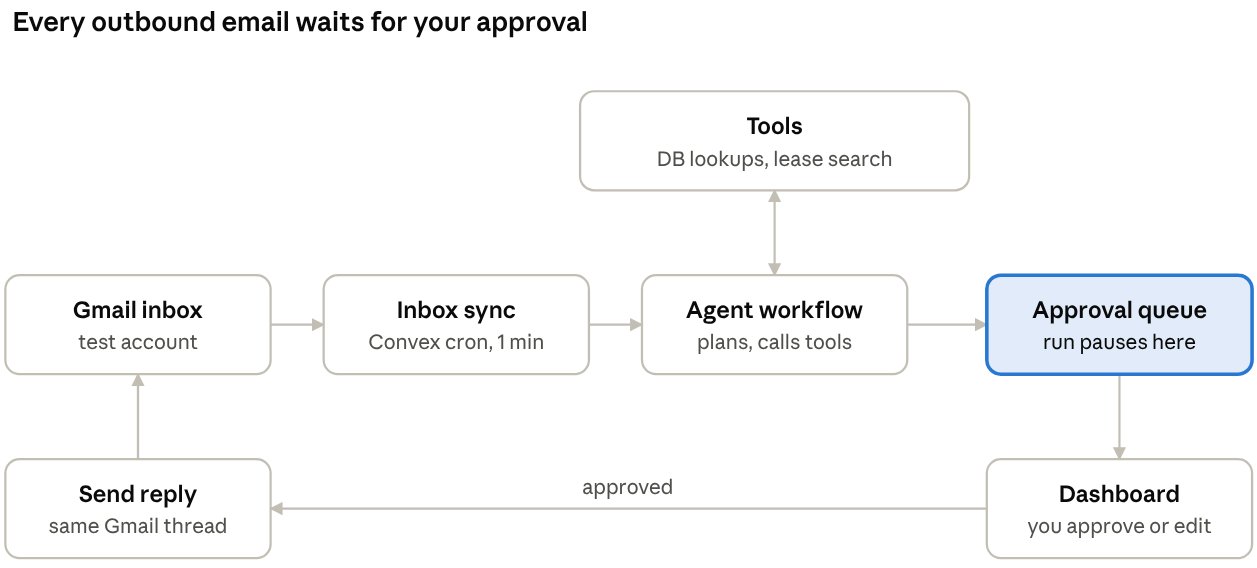
Layer	Choice	Why	Cost
Frontend	Next.js (App Router), React, Tailwind, shadcn/ui	Same as the target team's stack	Free
Backend + database	Convex: queries, mutations, actions, crons, file storage, vector search	One reactive backend, live UI updates for free	Free tier: 1M function calls/month, 0.5 GB database, 1 GB files (source)
Durable jobs	Convex Workflow + Workpool components	Retries, batching, runs that survive failures (docs)	Free
Auth	Convex Auth (password or GitHub)	Built into Convex	Free
LLM	Vercel AI SDK + Google Gemini	Tool calling and structured output; switching to Claude later is one line	Free tier, rate-limited. Paid only if limits are exceeded
Embeddings	Gemini embedding model	Same provider and key	Free tier
Email in and out	Gmail API on a dedicated test Gmail account	Real inbox, real threads	Free; needs a Google Cloud project (free)

Layer	Choice	Why	Cost
PDF text	unpdf	Runs in Convex Node actions	Free
CSV parsing	Papa Parse	Streams large files	Free
Validation	zod + Convex validators	Every tool input and import row is checked	Free
Hosting	Vercel + Convex cloud	Zero config deploys	Free
Testing	Vitest + convex-test	Unit tests on backend functions	Free

Paid items: none for the MVP. Stop and decide before moving past the Gemini or Convex free limits.

Architecture

A Convex cron pulls new Gmail messages, each one starts a durable agent workflow, and the workflow stops at the approval queue until you decide in the dashboard.



FlowPilot architecture · email in, agent run, approval, reply out

The dashboard is a Next.js app subscribed to Convex, so run steps, approvals and import progress update live with no polling code.

Data model

Fourteen Convex tables in three groups: property data, email and agent runs, background jobs. Every lookup the agent or UI makes goes through an index, never a full table scan.

Table	Key fields	Relations	Indexes
buildings	name, address, city	has many units	by_name
units	label, floor, sizeSqm, rent	belongs to building	by_building, by_building_label
tenants	name, email, phone	lives in a unit	by_email, by_unit
leases	startDate, endDate, rent, noticePeriodDays, status	tenant + unit, optional document	by_tenant, by_unit_status
tickets	category, priority, description, status	unit, tenant, source agent run	by_status, by_unit
documents	storageId, filename, kind, status, text	optional tenant or lease	by_status, by_tenant
docChunks	text, chunkIndex, embedding	belongs to document	vector index on embedding (filter by tenant)
emailThreads	subject, gmailThreadId, status, lastMessageAt	optional tenant	by_gmail_thread, by_last_message
emailMessages	direction, from, to, subject, body, gmailMessageId, receivedAt	belongs to thread	by_thread, by_gmail_message (dedupe)
agentRuns	status, category, urgency, summary, tokensIn, tokensOut, costUsd, startedAt, finishedAt	one per inbound message	by_status, by_message, by_thread
agentSteps	index, type (llm or tool), toolName, input, output, status, durationMs, error	belongs to run	by_run_index

Table	Key fields	Relations	Indexes
approvals	action, payload (draft), status, decidedAt, rejectReason	run + step	by_status, by_run
importJobs	kind, status, total, processed, failed, errors	uploaded file	by_status
users	Convex Auth tables	—	managed by Convex Auth

Design decisions to explain in interviews:

- Natural keys for upserts (tenant email, building + unit label), so re-running an import never creates duplicates.
- gmailMessageId is unique per message, so polling the inbox twice is safe.
- Steps live in their own table, so the run timeline is a small indexed query that updates live.
- importJobs keeps running counters, so the progress bar reads one row, not thousands.

Agent design

One inbound email starts one durable workflow run; read-only tools run freely, and any tool with a side effect pauses the run until a human decides.

The run, step by step

1. **Trigger:** a new inbound emailMessage starts a workflow and creates an agentRun.
2. **Classify:** one LLM call with structured output: category (repair, payment, lease question, move in or out, complaint, other), urgency, language, one-line summary.
3. **Gather context:** find the tenant by sender email, load unit, active lease and open tickets, vector-search the lease chunks with the email's question.
4. **Act:** an LLM tool loop, at most 8 tool calls, that ends in a draft reply and optionally a ticket proposal.
5. **Pause:** each side-effect tool creates an approval row and the workflow waits.
6. **Resume:** Approve executes it, Edit executes the edited version, Reject closes the run with the reason.

Tool registry. Each tool has a name, a description for the model, a zod input schema, a handler and a requiresApproval flag. The model only sees name, description and schema.

Tool	What it does	Side effect	Needs approval
find_tenant_by_email	Tenant, unit and building for a sender	No	No
get_lease	Active lease terms for a tenant	No	No
search_lease_documents	Vector search over that tenant's lease chunks	No	No
list_open_tickets	Open tickets for a unit	No	No
draft_reply	Saves a draft reply on the run	Draft only	No
escalate_to_human	Flags the thread as needing a person	Flag only	No
create_ticket	Opens a repair ticket	Writes data	Yes
send_reply	Sends the reply in the Gmail thread	External	Yes

Guardrails

- Max 8 tool calls and a token cap per run; over the cap, the run escalates.
- LLM and Gmail calls retry 3 times with backoff.
- Every tool input is validated with zod before the handler runs.
- An unknown sender never sees tenant data; the agent drafts a polite request to identify themselves.
- The reply is written in the language of the incoming email.
- Nothing is sent or created without an approval row marked approved.

Logging. Every LLM call and tool call writes an agentSteps row: input, output, duration, tokens, error. Run totals (tokens, cost in USD) are stored on agentRuns, and the dashboard subscribes to both live.

Build stages

Eight stages, each ending in something that runs and can be tested; do not start a stage until every box in the previous one is ticked.

Stage 0: Setup

Goal: an empty app running locally and deployed.

- Create the Next.js app with Tailwind and shadcn/ui, initialise Convex, add Convex Auth with one login method.
- Write CLAUDE.md: stack, folder layout, rules (strict TypeScript, zod on every input, no secrets in code, small commits).
- GitHub repo, deploy frontend to Vercel and backend to Convex production.

Done when:

- [ ] Login works on the deployed URL
- [ ] Local dev runs with no errors or type warnings

Stage 1: Property data

Goal: the data the agent will look up.

- Convex schema for buildings, units, tenants, leases and tickets, with the indexes from the data model.
- Seed script with realistic fake data: 3 buildings, about 60 units and tenants, one lease each.
- Pages: buildings list, building detail with units, tenant detail with lease and tickets.

Done when:

- [ ] The seed can run twice without duplicates
- [ ] Tenant lookup by email uses the by_email index

Stage 2: Bulk import

Goal: import thousands of records in the background without losing or duplicating any.

- Upload a CSV to Convex storage, create an importJob, parse it, process batches of 100 rows through Workpool.
- Validate each row with zod; bad rows go to the job's error list and the rest continue.

- Upsert by natural key; live progress bar (processed, failed, total) and a downloadable error report.
- A script that generates a 10,000-row test CSV.

Done when:

- [ ] 10,000 rows import with live progress, and re-importing the same file adds nothing
- [ ] A row with a broken email appears in the error report while the others import

Stage 3: Documents and knowledge base

Goal: lease PDFs become searchable for the agent.

- Multi-file PDF upload; one Workpool job per file: extract text, split into chunks with overlap, embed, save docChunks.
- Link each document to a tenant or lease (by filename pattern, or picked by hand).
- A search box on the dashboard to test vector search.
- A script that generates sample lease PDFs for the seed tenants.

Done when:

- [ ] 50 PDFs process in the background with a status per file
- [ ] Searching "notice period" for a tenant returns the right chunk of their lease

Stage 4: Email in and out

Goal: real emails flow through Gmail in both directions.

- Google Cloud project, Gmail API OAuth for the test inbox, tokens stored server side in Convex.
- A Convex cron every minute fetches new messages, dedupes by gmailMessageId, groups by Gmail thread and links the sender to a tenant.
- A sendReply action that answers inside the same thread.
- Inbox page with threads and messages, plus a "simulate email" form for demos.

Done when:

- [ ] An email sent to the test inbox shows in the dashboard within about a minute
- [ ] A reply sent from the app lands in the same Gmail thread

Stage 5: Agent engine

Goal: every new email starts a durable agent run.

- The tool registry and the read-only tools from Agent design.
- Workflow: classify, gather context, tool loop (max 8 calls), draft.
- agentRuns and agentSteps written at every step, with tokens and cost.
- Prompts kept as files in the repo so changes are versioned.

Done when:

- [ ] A repair email gets a category, the tenant's context and a draft reply
- [ ] A forced tool failure is retried and the run still completes

Stage 6: Approvals and run timeline

Goal: nothing leaves the system without a human click.

- create_ticket and send_reply create approvals and pause the workflow.
- Run page: live timeline of each step with input, output, duration and status.
- Approval card with Approve, Edit then approve, Reject with reason; the workflow resumes on the decision.
- Approvals queue page with a pending count in the nav.

Done when:

- [ ] Approve sends, Edit sends the edited text, Reject sends nothing
- [ ] The timeline updates live without a page refresh

Stage 7: Quality, deploy, portfolio

Goal: a demo you can show and defend in an interview.

- Eval set of 30 sample emails with expected category and action; a script reports accuracy.
- Loading, empty and error states across the UI; handling for LLM rate limits.
- Cost per run shown on the dashboard.
- Production deploy, README (architecture, schema, decisions, eval results) and a 2 to 3 minute demo video in English.

Done when:

- [ ] Eval accuracy is written in the README

- [ ] Someone else can open the demo, send a test email and see it handled

Non-functional requirements

The app should behave like production software, not a demo script: safe to retry, cheap to run, and closed to anyone who is not logged in.

Area	Requirement
Reliability	Every workflow step is idempotent; imports and inbox sync can run twice with the same result; failures are recorded with a reason
Performance	All lookups use indexes; long lists are paginated; no query reads a whole table
Cost	Token cap per run; tokens and cost logged per run and per step
Security	Secrets only in Convex environment variables; every query and mutation checks auth; email HTML is never rendered raw
Privacy	The agent shares tenant data only with the matching sender email
Testing	convex-test unit tests for imports, dedupe and approvals; the eval set for agent quality
Observability	Each run's steps, errors and timings are visible in the dashboard

Portfolio and working method

The deliverable is not only the app: it is the app plus the proof of how it was built.

Deliverables

- Live demo URL with a demo login
- Public GitHub repo with a README in English: architecture diagram, schema, key decisions, eval results, cost per run

- A 2 to 3 minute demo video in English: email arrives, agent run timeline, approval, reply in Gmail
- docs/decisions.md: one short entry per technical choice, used to prepare for interviews

How to build each stage with Claude Code

1. Keep CLAUDE.md current: stack, folder layout, rules, and which stage is in progress.
2. Start each stage in a new session: paste that stage's section from this PRD and ask for a plan before any code.
3. Review the plan, then let it build in small steps with a commit per feature.
4. Test every "Done when" box yourself before moving on.
5. Add one entry to docs/decisions.md per stage.

Prompt to start a stage:

```
Read CLAUDE.md. We are starting Stage 2: Bulk import from the PRD below.
First give me a plan: files to create or change, schema changes, and how you
will test each "Done when" item.
Do not write code until I approve the plan.

<paste the Stage 2 section here>
```

Risks and open questions

The biggest risks are free-tier limits and Gmail OAuth; both have a fallback.

Risk	Fallback
Gemini free-tier rate limits slow down evals and demos	Small model for classification, retries with backoff, run evals in batches
Google OAuth apps in testing mode expire refresh tokens after about 7 days	A "reconnect Gmail" button, and the simulate-email form for demos
Convex free quotas	A one-minute inbox cron is about 43,000 runs a month, well under 1M; watch the usage page
Scope creep (WhatsApp, voice, more tools)	Nothing new until Stage 7 is done

Open questions:

- [ ] Tenant emails: English only for the MVP, or also French and German?
- [ ] Public demo: shared demo login, or read-only demo mode?